

1.4 Word Embeddings and Semantic Geometry

Dense vectors, skip-gram, analogies, and why geometry can encode bias.

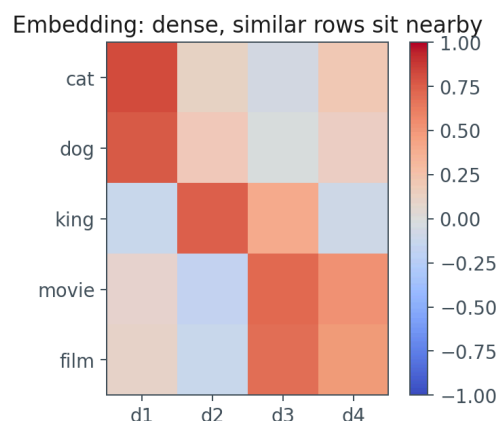
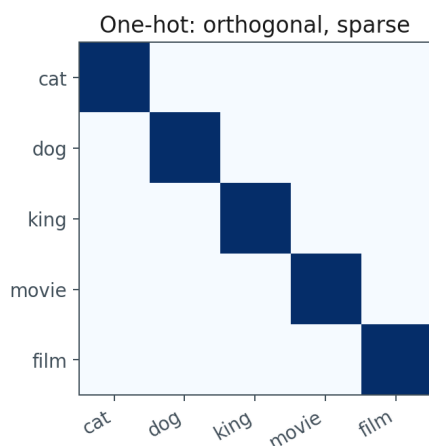
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A language model does not read characters as characters. It reads **vectors**. This week those vectors are word embeddings. From Week 3 they are token embeddings inside a transformer. The geometry is the same idea.

1. One-hot is a bad geometry

Give each word its own coordinate. *film* and *movie* are then orthogonal: inner product 0. The vector is sparse, as long as the vocabulary, and a new word has no coordinate at all.

A **distributed** representation is short and dense (50–300 dimensions is a classical size; transformer hidden sizes are larger). Meaning is spread across coordinates. Similar usage → similar vectors.



Distributional is about the data: words that occur in similar contexts mean similar things (Harris).

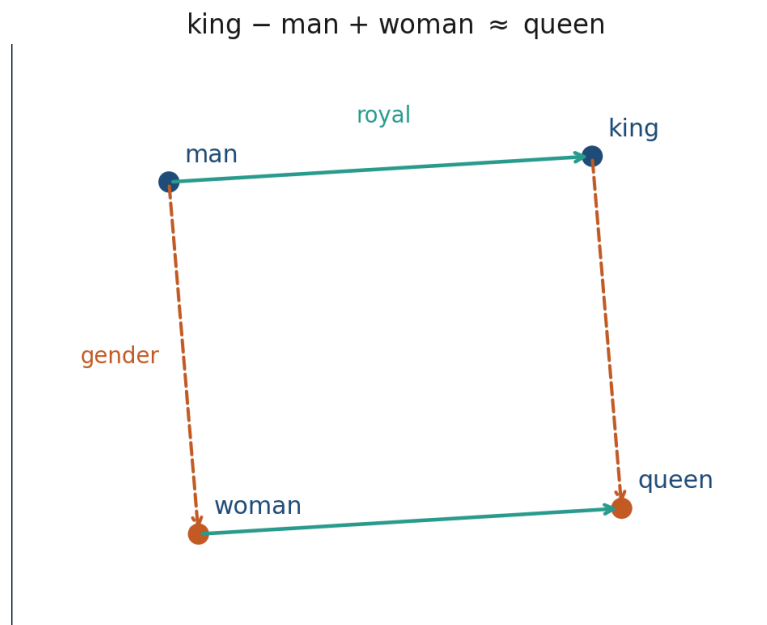
Distributed is about the code: a dense vector. An **embedding** is the map from a token id into that vector.

2. Semantic geometry

If the map is good, **direction means something**. The Mikolov analogy is the slogan:

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}.$$

The same offset can look like gender, tense, or capital-of. You measure similarity with **cosine**, not Euclidean distance (length is partly frequency).



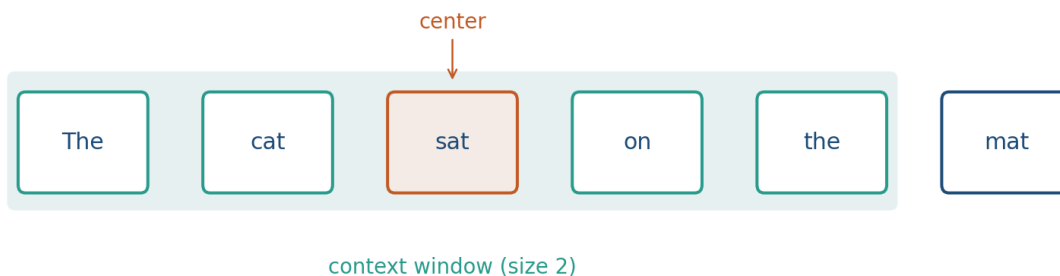
Nearest neighbors are a debug tool. If *Paris* sits next to *France* and *Rome*, training learned something. If it sits next to punctuation, the corpus or the window is wrong.

3. How Word2Vec learns the map

You do not build a huge co-occurrence matrix and then factor it (that is LSA / GloVe). **Word2Vec** trains a small two-layer net on a fake task.

Skip-gram: from the center word, predict each neighbor in a window. **CBOW**: from the neighbors, predict the center.

Skip-gram: predict each neighbor from the center word



Input is a one-hot (or an id lookup, which is the same map). The hidden layer has width d . After training you **throw away the softmax** and keep the hidden weights. Those rows are the embeddings. A full softmax over 100,000 words is slow. Negative sampling (a later trick) replaces it. You do not need that trick for Lab 1.

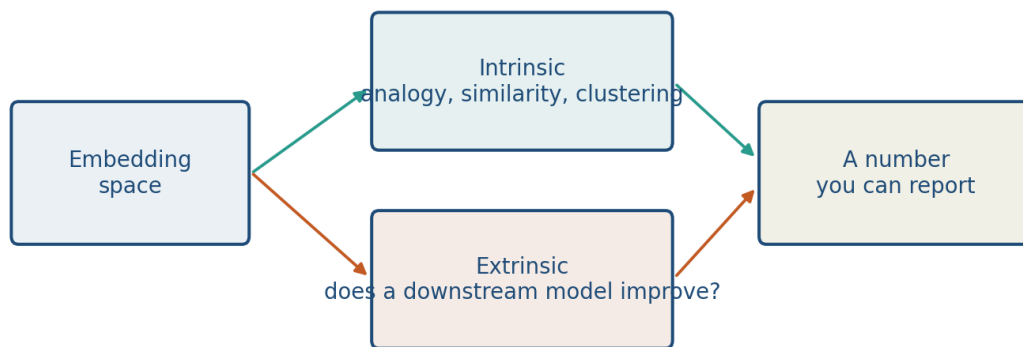
A transformer does not throw the rest of the net away: it *keeps* transforming those vectors. Word2Vec is the geometry without the rest of the stack.

4. Intrinsic versus extrinsic evaluation

Intrinsic: does the space look right on its own? Analogies, word similarity datasets, clustering of synsets.

Extrinsic: plug the vectors into a downstream model (sentiment, retrieval, NER). If the task score moves, the embedding helped that task.

Two ways to score embeddings. They need not agree.

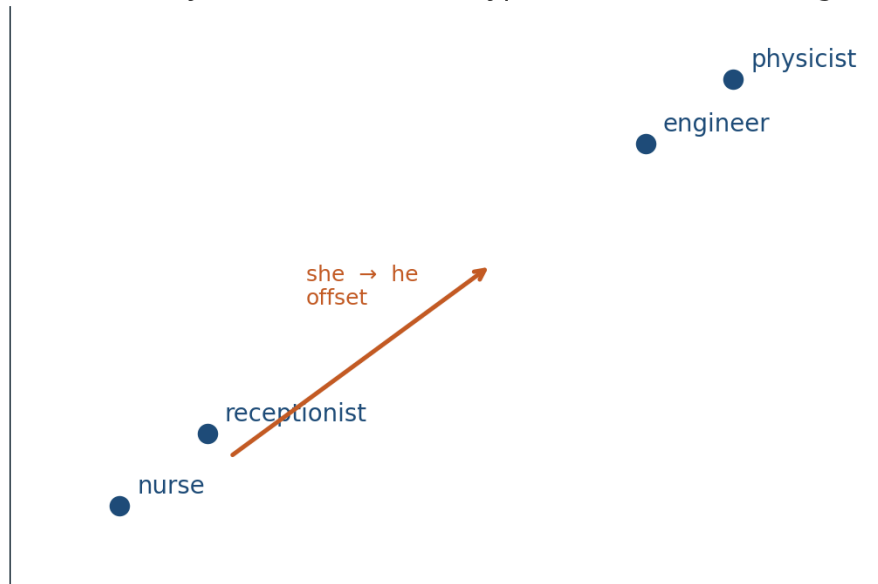


They need not agree. A space that wins analogies can still be a weak feature for your classifier. For a project, extrinsic is the number that matters. Intrinsic is how you debug.

5. Bias is also geometry

The same offsets that encode "capital of" can encode stereotypes. Occupation words often sit along a she/he direction in older embeddings. That is not a bug in cosine; it is a property of the training text.

Geometry can encode stereotypes as well as meaning



You will measure this in Lab 1 on a tiny, constructed space, then later on real models. Debiasing is incomplete; reporting the probe is part of the project writeup when your system ranks people, jobs, or medical outcomes.

6. Practice

1. In one sentence each: distributional, distributed, embedding.
2. Why is cosine the default similarity?
3. Give one intrinsic and one extrinsic test for a word embedding. Which one would you put in a project report?